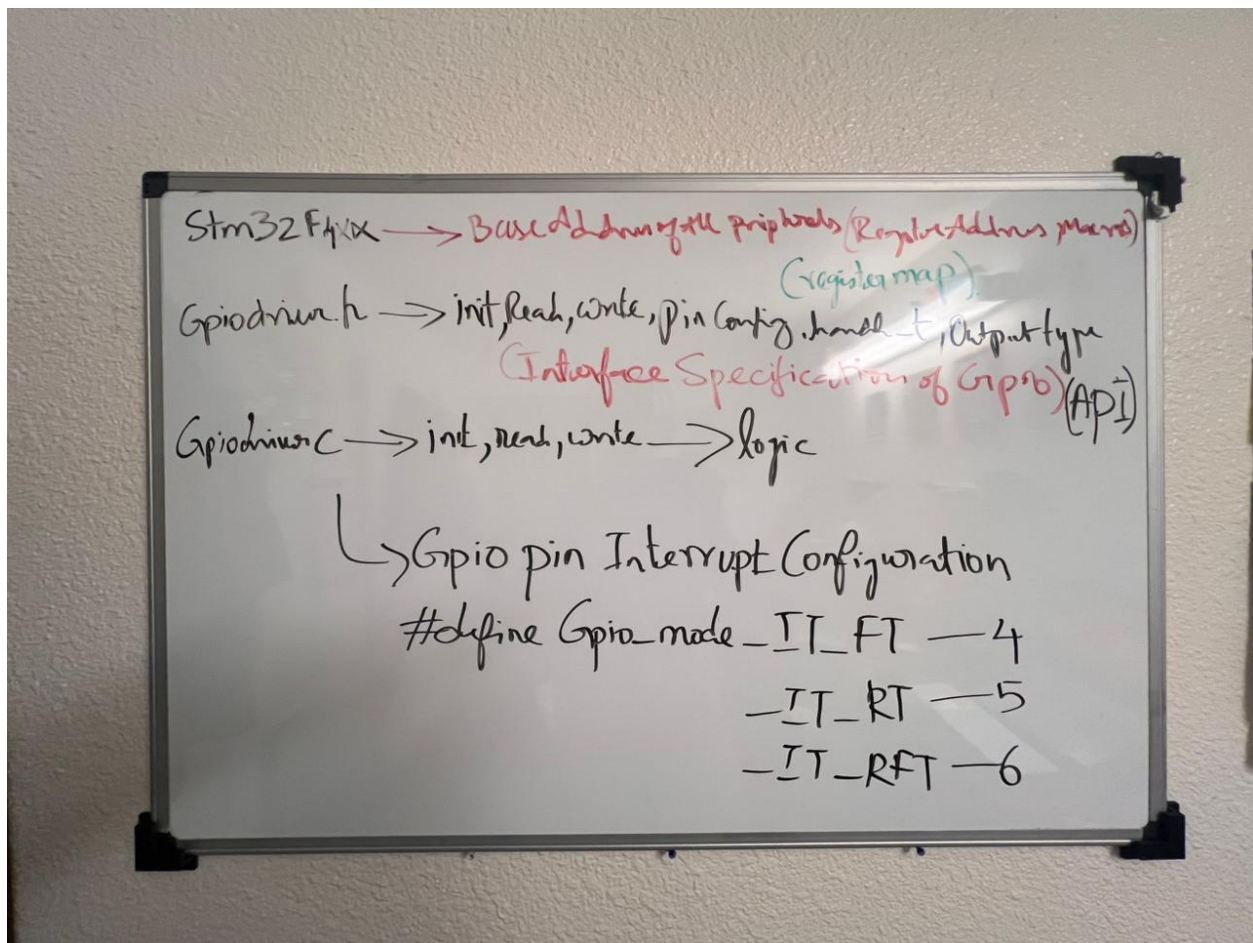




Driver layering + where GPIO interrupt config fits (Whiteboard)

This diagram captures the **overall bare-metal driver workflow** I'm following for STM32:

- **stm32f4xx.h (base addresses + peripheral register definitions):** Defines memory map constants and C structs for registers (RCC/GPIO/EXTI/SYSCFG/NVIC). This is the hardware "truth layer".
- **GPIO driver (GPIO init/read/write/toggle + interrupt config):** Implements the *GPIO API* that application code calls. This file contains the logic that writes to the MCU registers.
- **Application (main.c):** Uses the GPIO API without touching registers directly.

This image also introduces the **GPIO interrupt pin modes** (interrupt on falling edge, rising edge, or both). These modes are encoded as macros (example: GPIO_MODE_IT_FT,

GPIO_MODE_IT_RT, GPIO_MODE_IT_RFT) and are used inside GPIO_Init() to decide which EXTI trigger registers to program.

```
{
    //this is used for interrupt (interrupt mode)
    if (pGPIOHandle->GPIO_PinConfig.GPIO_PinMode == GPIO_MODE_IT_FT)
    {
        //1. configure the FTSR
        EXTI->FTSR |= (1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
        //2. clear the corresponding RTSR bit
        EXTI->RTSR &= ~(1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
    }
    else if (pGPIOHandle->GPIO_PinConfig.GPIO_PinMode == GPIO_MODE_IT_RT)
    {
        //1.configure the RTSR
        EXTI->RTSR |= (1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);

        //2 clear the corresponding FTSR bit
        EXTI->FTSR &= ~(1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
    }
    else if (pGPIOHandle->GPIO_PinConfig.GPIO_PinMode == GPIO_MODE_IT_RFT)
    {
        //1.configure both FTSR and RTSR
        EXTI->FTSR |= (1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
        EXTI->RTSR |= (1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
    }
    //2.configure the GPIO port selection in SYSCFG_EXTICR
    uint8_t temp1 = pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber / 4;
    uint8_t temp2 = pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber % 4;
    uint8_t portcode = GPIO_BASEADDR_TO_CODE(pGPIOHandle->pGPIOx);
    SYSCFG_PCLK_EN();
    SYSCFG->EXTICR[temp1] = portcode << (temp * 4);

    //3.enable the EXTI interrupt delivery using IMR (interrupt mask register)
    EXTI->IMR |= (1 << pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber);
}
```

EXTI trigger selection + SYSCFG mapping + IMR enable (Code block)

This code block shows the **core of GPIO interrupt configuration** when the pin is configured in an interrupt mode.

1) Trigger selection (FTSR / RTSR)

Depending on the selected mode:

- **Falling edge:** set the bit in **FTSR**, clear in **RTSR**

- **Rising edge:** set the bit in **RTSR**, clear in **FTSR**
- **Rising + Falling:** set the bit in **both FTSR and RTSR**

This matters because EXTI must know **what edge produces an interrupt request** for a given EXTI line number (0–15).

2) Port selection using SYSCFG_EXTICR

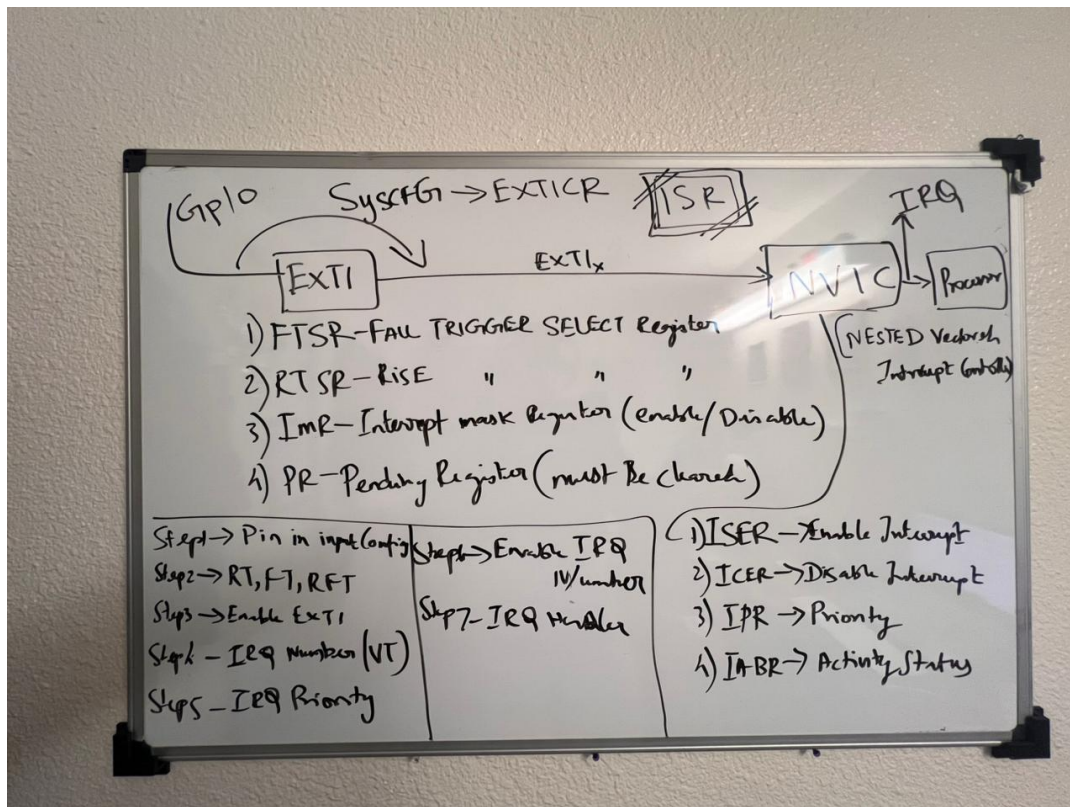
Even if EXTI line number matches the pin number, EXTI still needs to know **which GPIO port** (A/B/C/...) is connected to that EXTI line.

That routing is done through **SYSCFG → EXTICR[x]** fields.

3) Interrupt unmask (IMR)

Finally:

- **IMR (Interrupt Mask Register)** is set for that line to **allow the interrupt request to propagate**.
If IMR is not set, the edge can occur and flags may update, but the interrupt won't reach the CPU.



Full signal path: GPIO → SYSCFG → EXTI → NVIC → ISR (Whiteboard)

This board is the “big picture” interrupt flow:

1. **GPIO pin toggles (external event)**
2. **SYSCFG_EXTICR** decides which port is connected to EXTI line (e.g., PB4 → EXTI4)
3. **EXTI trigger logic** decides whether the event is valid (FTSR/RTSR)
4. **EXTI pending flag (PR)** is set when a valid trigger happens
5. If **IMR is unmasked**, an **IRQ request** is generated
6. **NVIC** decides whether the CPU will run the ISR (based on enable + priority)
7. **ISR executes**, then software clears **PR** to acknowledge the interrupt

Key registers called out on the board:

- **FTSR / RTSR**: choose falling/rising triggers
- **IMR**: enable/disable interrupt delivery
- **PR**: pending flag; must be cleared to stop repeated interrupts
- **NVIC ISER/ICER**: enable/disable IRQ at CPU interrupt controller level

0x4001 3C00 - 0x4001 3FFF	EXTI	APB2	Section 12.3.7: EXTI register map on page 390
0x4001 3800 - 0x4001 3BFF	SYSCFG		Section 9.2.8: SYSCFG register maps for STM32F405xx/07xx and STM32F415xx/17xx on page 297 and Section 9.3.8: SYSCFG register maps for STM32F42xxx and STM32F43xxx on page 304

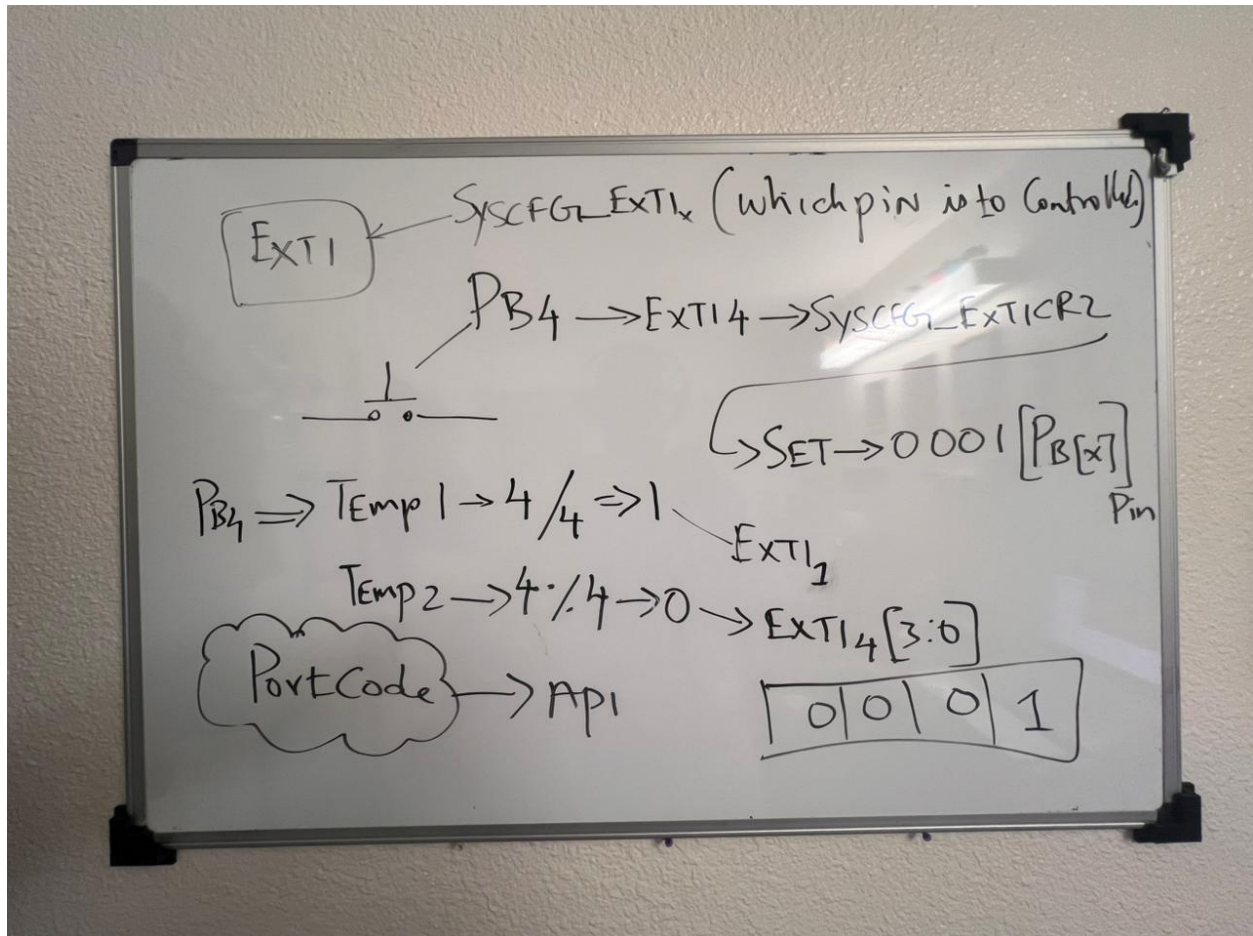
SYSCFG register map → why we create SYSCFG struct (Reference screenshot + typedef)

This image documents **why the SYSCFG register struct exists** in your header:

- The STM32 reference manual lists **SYSCFG registers** and their offsets (EXTICR1..4 are the key ones for EXTI routing).
- Your typedef struct { ... } SYSCFG_RegDef_t; mirrors that register map so you can write:
 - SYSCFG->EXTICR[x] = ... instead of hardcoding addresses.

Reason for implementation:

This makes the code **readable, maintainable, and less error-prone**, because register offsets are represented structurally rather than manually computed each time.



EXTICR math example: PB4 → EXTI4 → EXTICR2 (Code + Whiteboard)

This section explains *how the EXTICR index and bit field are calculated*.

Why the division and modulo?

Each **EXTICR** register holds **4 EXTI** lines, and each line consumes **4 bits**:

- $\text{temp1} = \text{PinNumber} / 4 \rightarrow \text{which EXTICR register (0..3)}$
- $\text{temp2} = \text{PinNumber} \% 4 \rightarrow \text{which 4-bit slot inside that EXTICR}$

Example shown: **PB4**

- **Pin = 4**

- $\text{temp1} = 4/4 = 1 \rightarrow \text{EXTICR2}$ (index 1)
- $\text{temp2} = 4\%4 = 0 \rightarrow \text{slot 0}$ (bits [3:0]) corresponds to EXTI4

Port code

$\text{portcode} = \text{GPIO_BASEADDR_TO_CODE}(\text{GPIOx})$ returns a small number:

- A=0, B=1, C=2, ...
So PB4 uses **portcode = 1**, and the code writes that into the EXTI4 field:
- $\text{SYSCFG} \rightarrow \text{EXTICR}[\text{temp1}] |= (\text{portcode} \ll (\text{temp2} * 4));$

Reason for implementation:

This generalizes routing so any pin (0–15) on any port can be mapped without manual register editing.

9.2.8 SYSCFG register maps for STM32F405xx/07xx and STM32F415xx/17xx

The following table gives the SYSCFG register map and the reset values.

**Table 41. SYSCFG register map and reset values
(STM32F405xx/07xx and STM32F415xx/17xx)**

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	SYSCFG_MEMRMP	Reserved																														MEM_MODE	
	Reset value																																x
0x04	SYSCFG_PMC	Reserved				0	MII_RMII_SEL	Reserved				Reserved																					
	Reset value																																
0x08	SYSCFG_EXTICR1 Reset value	Reserved										EXTI3[3:0] 0 0 0 0			EXTI2[3:0] 0 0 0 0			EXTI1[3:0] 0 0 0 0			EXTI0[3:0] 0 0 0 0												
0x0C	SYSCFG_EXTICR2 Reset value	Reserved										EXTI7[3:0] 0 0 0 0			EXTI6[3:0] 0 0 0 0			EXTI5[3:0] 0 0 0 0			EXTI4[3:0] 0 0 0 0												
0x10	SYSCFG_EXTICR3 Reset value	Reserved										EXTI11[3:0] 0 0 0 0			EXTI10[3:0] 0 0 0 0			EXTI9[3:0] 0 0 0 0			EXTI8[3:0] 0 0 0 0												
0x14	SYSCFG_EXTICR4 Reset value	Reserved										EXTI15[3:0] 0 0 0 0			EXTI14[3:0] 0 0 0 0			EXTI13[3:0] 0 0 0 0			EXTI12[3:0] 0 0 0 0												
0x20	SYSCFG_CMPCR	Reserved														READY 0	Reserved						CMP_PD										
	Reset value																																

Refer to [Section 2.3: Memory map](#) for the register boundary addresses.

```

3 /*****Peripheral definitions for Peri_sysconfig *****/
4 typedef struct
5 {
6     _vo uint32_t MEMRMP; /* !< give a short description          address offset 0x00 */
7     _vo uint32_t PMC; /* !< TODO,                                address offset 0x04 */
8     _vo uint32_t EXTICR[4]; /* !< TODO                      address offset 0x08 - 0x14 */
9     uint32_t RESERVED1[2]; /* !< TODO                      address offset 0x18 - 0x1C */
10    _vo uint32_t CMPR; /* !< TODO                      address offset 0x20 */
11    uint32_t RESERVED2[2]; /* !< TODO                      address offset 0x24 - 0x28 */
12    _vo uint32_t CFGR; /* !< TODO                      address offset 0x2C */
13 } SYSCFG_RegDef_t;
14
15 /*****Peripheral definitions typed base addresses to xxx_RegDef*****/
16
17 #define RCC ((RCC_RegDef_t*)RCC_BASEADDR)
18
19
20 //2.configure the GPIO port selection in SYSCFG_EXTICR
21 uint8_t temp1 =pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber /4;
22 uint8_t temp2 =pGPIOHandle->GPIO_PinConfig.GPIO_PinNumber %4;
23 uint8_t portcode =GPIO_BASEADDR_TO_CODE(pGPIOHandle->pGPIOx);
24 SYSCFG_PCLK_EN();
25 SYSCFG->EXTICR[temp1]= portcode << (temp2 *4);
26
27
28
29
30 #define GPIO_BASEADDR_TO_CODE(x) ( (x==GPIOA) ? 0 :\
31                                     (x==GPIOB) ? 1 :\
32                                     (x==GPIOC) ? 2 :\
33                                     (x==GPIOD) ? 3 :\
34                                     (x==GPIOE) ? 4 :\
35                                     (x==GPIOF) ? 5 :\
36                                     (x==GPIOG) ? 6 :\
37                                     (x==GPIOH) ? 7 :0 )
38
39 /*

```

NVIC Configuration

```

13  /*
14  * Processor specific details
15  * Arm CORTEX Mx Processor NVIC ISERx register addresses
16  */
17  #define NVIC_ISER0      ((_vo uint32_t*)0xE000E100)
18  #define NVIC_ISER1      ((_vo uint32_t*)0xE000E104)
19  #define NVIC_ISER2      ((_vo uint32_t*)0xE000E108)
20  #define NVIC_ISER3      ((_vo uint32_t*)0xE000E10C)
21
22  /*
23  * Arm CORTEXMx processor NVIC ICERx register addresses
24  */
25  #define NVIC_ICER0      ((_vo uint32_t*)0xE000E180)
26  #define NVIC_ICER1      ((_vo uint32_t*)0xE000E184)
27  #define NVIC_ICER2      ((_vo uint32_t*)0xE000E188)
28  #define NVIC_ICER3      ((_vo uint32_t*)0xE000E18C)
29
30

```

GPIO_IRQInterruptConfig(): mapping IRQ number to ISER/ICER (Code)

This function is the software wrapper that **enables/disables an IRQ number** inside NVIC.

What the logic is doing

- If EnorDi == ENABLE:
 - IRQ 0..31 → set bit in NVIC_ISER0
 - IRQ 32..63 → set bit in NVIC_ISER1 using (IRQNumber % 32)
 - IRQ 64..95 → set bit in NVIC_ISER2 using (IRQNumber % 64) (your code uses shifting with (IRQNumber - 64) style)
- Else (DISABLE):
 - Same idea but using **ICERx** registers

Reason for implementation:

This keeps NVIC enable/disable consistent and avoids repeated magic numbers across the project. It also matches the hardware layout: **32 IRQs per register**.


```

49 void GPIO_IRQInterruptConfig(uint8_t IRQNumber, uint8_t EnorDi) //IRQ config and handling
50 {
51     if (EnorDi == ENABLE)
52     {
53         if (IRQNumber <= 31)
54         {
55             //program for ISER0 register
56             *NVIC_ISER0 |= (1 << IRQNumber );
57
58         }else if (IRQNumber > 31 && IRQNumber <64) //32 to 63
59         {
60             //program for ISER1 register
61             *NVIC_ISER1 |= (1 << (IRQNumber % 32) );
62         }else if (IRQNumber >= 64 && IRQNumber <96) //64 to 95
63         {
64             //program for ISER2 register
65             *NVIC_ISER2 |= (1 << (IRQNumber % 64) );
66         }
67     }else
68     {
69         if (IRQNumber <= 31)
70         {
71             //program ICER0 Register
72             *NVIC_ICER0 |= (1 << IRQNumber );
73         }else if (IRQNumber > 31 && IRQNumber <64) //32 to 63
74         {
75             //program for ISER1 register
76             *NVIC_ICER1 |= (1 << (IRQNumber % 32) );
77         }else if (IRQNumber >= 64 && IRQNumber <96) //64 to 95
78         {
79             //program for ISER2 register
80             *NVIC_ICER2 |= (1 << (IRQNumber % 64) );
81         }
82     }
}

```

NVIC enable/disable registers (ISER/ICER) (NVIC screenshot)

This image documents how **NVIC enables/disables interrupts**:

- **ISERx** registers: writing 1 enables an IRQ number
- **ICERx** registers: writing 1 disables an IRQ number
- Each ISER/ICER register covers **32 IRQs**, so IRQ numbers are grouped:
 - 0–31 → ISER0 / ICER0
 - 32–63 → ISER1 / ICER1
 - 64–95 → ISER2 / ICER2
 - etc.

Reason for implementation:

Even if EXTI generates an interrupt request, the CPU won't run the ISR unless the corresponding IRQ is **enabled in NVIC**.

4.2.2 Interrupt Set-enable Registers

The NVIC_ISER0-NVIC_ISER7 registers enable interrupts, and show which interrupts are enabled. See the register summary in [Table 4-2 on page 4-3](#) for the register attributes.

The bit assignments are:



Table 4-4 ISER bit assignments

Bits	Name	Function
[31:0]	SETENA	Interrupt set-enable bits. Write: 0 = no effect 1 = enable interrupt. Read: 0 = interrupt disabled 1 = interrupt enabled.

If a pending interrupt is enabled, the NVIC activates the interrupt based on its priority. If an interrupt is not enabled, asserting its interrupt signal changes the interrupt state to pending, but the NVIC never activates the interrupt, regardless of its priority.

4.2.7 Interrupt Priority Registers

The NVIC_IPRO-NVIC_IPR59 registers provide an 8-bit priority field for each interrupt and each register holds four priority fields. These registers are byte-accessible. See the register summary in [Table 4-2 on page 4-3](#) for their attributes. Each register holds four priority fields as shown:

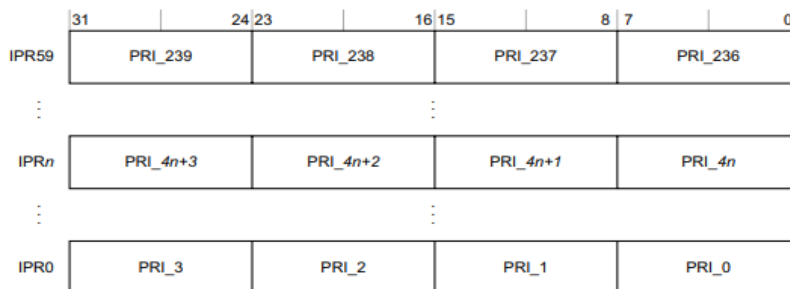


Table 4-9 IPR bit assignments

Bits	Name	Function
[31:24]	Priority, byte offset 3	Each implementation-defined priority field can hold a priority value, 0-255. The lower the value, the greater the priority of the corresponding interrupt. Register priority value fields are eight bits wide, and non-implemented low-order bits read as zero and ignore writes.
[23:16]	Priority, byte offset 2	
[15:8]	Priority, byte offset 1	
[7:0]	Priority, byte offset 0	

```

5
6 void GPIO_IRQPriorityConfig (uint8_t IRQNumber, uint8_t IRQPriority )
7 {
8     //1. First find the ipr register
9     uint8_t iprx = IRQNumber / 4;
10    uint8_t iprx_section = IRQNumber % 4 ;
11
12    uint8_t shift_amount = (8* iprx_section) + (8 - NO_PR_BITS_IMPLEMENTED);
13    *(NVIC_PR_BASE_ADDR + iprx * 4 ) |= (IRQPriority << shift_amount);
14 }
15
16
17

```

NVIC priority programming (IPR layout + GPIO_IRQPriorityConfig code)

This image shows how interrupt priorities are assigned using NVIC **IPR (Interrupt Priority Registers)**.

Key facts

- Each IRQ gets an **8-bit priority field**
- One IPR register holds **4 priority fields** (4 IRQs per register)
- Many STM32 parts implement only the upper bits (lower bits are not implemented), which is why your code uses:
 - $\text{shift_amount} = (8 * \text{section}) + (8 - \text{NO_PR_BITS_IMPLEMENTED})$

What your function computes

- $\text{iprx} = \text{IRQNumber} / 4 \rightarrow$ which IPR register
- $\text{iprx_section} = \text{IRQNumber} \% 4 \rightarrow$ which byte inside that register
- then it writes IRQPriority into the correct position.

Reason for implementation:

This gives deterministic control over which interrupt preempts others (critical for real-time behavior and for safety-style reasoning later).

```

297
298 void GPIO_IRQHandling(uint8_t PinNumber)
299 {
300     //clear the exti pr register corresponding to the pin number
301     if (EXTI ->PR & (1 <<PinNumber))
302     {
303         //clear
304         if (EXTI ->PR |= (1 <<PinNumber));
305     }
306 }
307

```

IRQ handling: check PR and clear PR (Code)

This is the **interrupt acknowledge step**.

What PR means

- **PR (Pending Register)** indicates that an EXTI line has a pending interrupt event.
- When an edge occurs (and is not masked), hardware sets the PR bit.

Why we clear it (and why by writing 1)

In STM32 EXTI, you typically clear the pending bit by **writing 1 to that bit** (write-to-clear behavior). Your handler:

- checks if PR bit is set for the pin
- then clears it so the ISR won't keep retriggering immediately

Reason for implementation:

If you don't clear PR, the interrupt can appear "stuck" and the CPU may keep re-entering the handler.